

ENGINEERING NOTEBOOK

WRO 2026 — Future Engineers

Team: **UWR Red Torque**

Entries EN-001 through EN-011
Project Kick-off → Pre-TF-Sensor Integration Phase

EN-001 – Project Kick-off

Date: 25 / 03 / 2026

OBJECTIVE

Initiate development of an autonomous vehicle for the WRO 2026 Future Engineers challenge. Focus: a reliable robot capable of autonomous navigation, wall following, obstacle detection, traffic sign handling, and parking – while remaining modular for future improvement.

INITIAL DISCUSSION

Team agreed to continue with the LEGO ecosystem for rapid prototyping. With the Open Challenge approaching, priority was a functional robot fast, not fully optimized subsystems.

INITIAL HARDWARE SELECTION

Component	Role
LEGO Technic chassis	Base structure
LEGO SPIKE Prime Hub	Motor / sensor control
LEGO motors	Drive + steering
Raspberry Pi	Vision processing
Raspberry Pi Camera Module 3 Wide	Color block / sign detection
ESP32	Comms bridge
2x LEGO Ultrasonic Sensors	Wall distance

Decision: use ultrasonic sensors already in the kit to save time/cost during the initial phase.



Fig. EN-001.1 – Initial system block diagram (rough plan, pre-assembly)

EN-002 — Mechanical Assembly Begins

Date: 27 / 03 / 2026

First chassis assembled from LEGO Technic components. Steering and drivetrain integrated; Raspberry Pi, ESP32, and camera mounted. Initial wiring completed to verify communication between the compute unit and robot hardware.

CURRENT STATUS

- ☐ Chassis assembled
- ☐ Steering functional
- ☐ Drive system operational
- ☐ Camera mounted
- ☐ Basic electronics integrated

Note: cable routing still messy at this stage — revisit mounting later (see EN-004).

ASSEMBLY NOTES

Steering linkage built using standard LEGO Technic steering rack geometry — single central motor driving both front wheels via a turntable link. Chosen for simplicity over independent-wheel steering, which would have needed two motors and more control logic.

Drivetrain uses a single geared motor connected to the rear axle through a small gear reduction, to trade top speed for more predictable low-speed control (important for tight arena turns).

Raspberry Pi mounted on a raised platform at the front-center of the chassis to keep the camera's line of sight clear. ESP32 mounted low and to the side, closer to the SPIKE Prime hub, to keep the serial/wired link between them short.

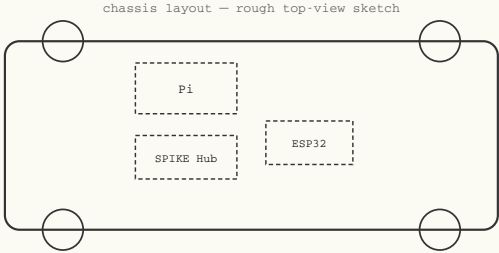


Fig. EN-002.1 — Rough component placement sketch on first assembled chassis

To check next session: motor mount screws loosened slightly after first test runs — add thread lock or lock washers.

EN-003 — First Autonomous Tests

Date: 29 / 03 / 2026

Software focus: basic autonomous movement. The two LEGO ultrasonic sensors configured for wall-distance measurement. PID controller implemented to keep the robot centered between arena walls.

Raspberry Pi processed camera images for color block detection. ESP32 acted as the communication interface between the Raspberry Pi and the LEGO control system.

OBSERVATIONS

Robot completed straight paths and basic turns successfully. Occasional steering oscillations and inconsistent distance readings observed — flagged for tuning.



rough sketch: steering oscillation pattern vs target centerline

Fig. EN-003.1 — Observed steering oscillation during wall-follow test (hand-sketch reference)

EN-004 — Open Challenge Preparation

Date: 31 / 03 / 2026

Objective shifted from optimization to achieving a stable, reliable robot for the Open Challenge. Multiple iterations of:

- PID tuning
- Steering calibration

Minor structural improvements made to sensor mounting and cable management (following note from EN-002).

PID TUNING LOG (INFORMAL)

Attempt	Kp	Ki	Kd	Result
1	1.0	0.0	0.0	Slow to react, drifts into wall on turns
2	2.2	0.0	0.4	Reacts faster, mild oscillation returns
3	1.8	0.05	0.6	Noticeably smoother, small overshoot on sharp corners

Settled on attempt 3 as the working baseline going into the Open Challenge, with the understanding that further tuning would likely be needed once new sensors were introduced later (see EN-008 onward).

EN-005 — Open Challenge Qualification

Date: 03 / 04 / 2026

Robot successfully completed the Open Challenge and qualified in the first week of April.

HARDWARE CONFIGURATION (AT QUALIFICATION)

Component	Position
LEGO Ultrasonic Sensor	Left
LEGO Ultrasonic Sensor	Right
Raspberry Pi	Onboard compute
ESP32	Comms bridge
Raspberry Pi Camera Module 3 Wide	Front, vision
LEGO SPIKE Prime	Motor control

ISSUES IDENTIFIED

- Wide sensing angle
- Reflection from nearby objects
- Inconsistent readings near corners
- Reduced accuracy at higher speeds
- Sensor noise affecting steering stability

Noted for future improvement after the Open Challenge.

RUN NOTES

Robot completed the required laps within the allotted attempts. Timing was not the strongest part of the run — most time lost during corner recovery when ultrasonic readings briefly spiked due to reflection off the inner wall angle.

No parking module attempted at this stage; the team decided to keep the Open Challenge submission focused purely on reliable lap completion rather than risk an unfinished feature.

TEAM REFLECTION

Overall the LEGO ultrasonic sensors did their job for a first qualification but were clearly the ceiling on how tight and fast the robot could safely run. This became the seed for the sensor study carried out later in EN-008.

| Photo/video reference of the qualifying run to be added here once available.

EN-006 — Development Pause & Resume

Paused: 05 / 04 / 2026 — Resumed: 15 / 06 / 2026

THE PAUSE

Following Open Challenge completion, development paused temporarily due to academic commitments and examination schedules. No major hardware or software modifications were made during this period, but observations and improvement ideas from the qualifying run were kept on a shared note rather than left to memory.

IMPROVEMENT BACKLOG (COMPILED DURING THE PAUSE)

- Investigate replacing ultrasonic sensors with a more precise distance-sensing technology
- Improve cable management around the SPIKE Prime hub — several wires still loose
- Revisit PID constants once new sensors are in place, since noise characteristics will differ
- Look into a proper parking maneuver, not attempted during the Open Challenge
- Consider adding a second camera angle if front-only vision proves limiting for sign detection

RESUMING WORK

Project work resumed with the objective of preparing for the final WRO competition. Rather than jumping straight back into software, the team first reviewed the overall robot architecture to find the changes with the greatest performance gain, and walked through the improvement backlog above as a group. Sensor replacement was agreed as the top priority, since most of the other items — PID tuning, parking — depend on having stable, accurate distance data to build on.

The existing robot was also briefly re-tested as-is, to re-familiarize with its current behavior after the break. Steering and drive were still functional; the ultrasonic inconsistency near corners was immediately noticeable again, confirming it as the right first target.

PLAN FOR THE COMING WEEKS

1. Research alternative distance sensors (EN-008)
2. Decide on architecture changes needed to support them (EN-009)
3. Order parts and plan integration (EN-010)
4. Physical integration and re-testing (EN-011 onward)

EN-008 — Sensor Technology Study

Date: 20 / 06 / 2026

Engineering review of alternative distance-sensing technologies.

SENSORS EVALUATED

Sensor	Type
TF-Luna	ToF LiDAR, single-point
TF-LC02	ToF LiDAR, short-range
VL53L0X	ToF LiDAR, short-range
VL53L1X	ToF LiDAR, long-range

COMPARISON CRITERIA

- Measurement accuracy
- Response time
- Beam width
- Reliability
- Cost
- Ease of integration
- Suitability for the WRO arena

Conclusion: Time-of-Flight (ToF) LiDAR sensors should significantly improve wall-following precision and obstacle detection over ultrasonic sensors.

NOTES ON INDIVIDUAL SENSORS

TF-Luna — single-point ToF, UART or I2C output, narrow beam angle, longer range. Good fit for side wall-following where a narrow, focused reading matters more than wide coverage.

TF-LC02 — shorter-range ToF, smaller form factor, good for close-range obstacle/parking detection at the front and rear where precision at short distance matters more than long range.

VL53L0X / VL53L1X — I2C ToF sensors, well documented, cheap, but each needs a unique I2C address or an XSHUT pin to multiplex multiple units on one bus — added wiring complexity noted as a downside versus the UART-based TF series.

DECISION RATIONALE

Chose a mixed TF-Luna / TF-LC02 setup over an all-VL53L0X setup mainly to avoid the I2C address-multiplexing wiring overhead under time pressure, and because the TF series' longer effective range suited the side wall-following use case better.

EN-009 — System Architecture Review

Date: 01 / 07 / 2026

Introducing four ToF sensors required reconsidering the electronics architecture. Three integration approaches discussed:

1. Connect all sensors directly to the Raspberry Pi
2. Shift complete sensor handling to the existing ESP32
3. Introduce a second ESP32 dedicated to sensor acquisition, retaining the existing communication architecture

With the competition approaching, minimizing software/hardware churn was an important constraint. Stability and modularity prioritized over a full redesign — option 3 favored.

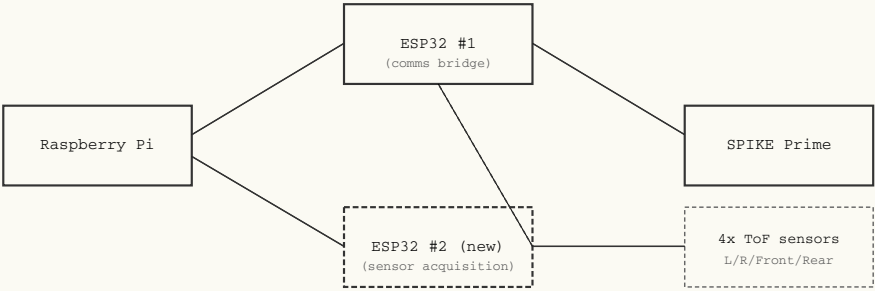


Fig. EN-009.1 — Chosen architecture: dedicated second ESP32 for sensor acquisition, existing Pi•ESP32#1•SPIKE link untouched

EN-010 — Procurement and Integration Planning

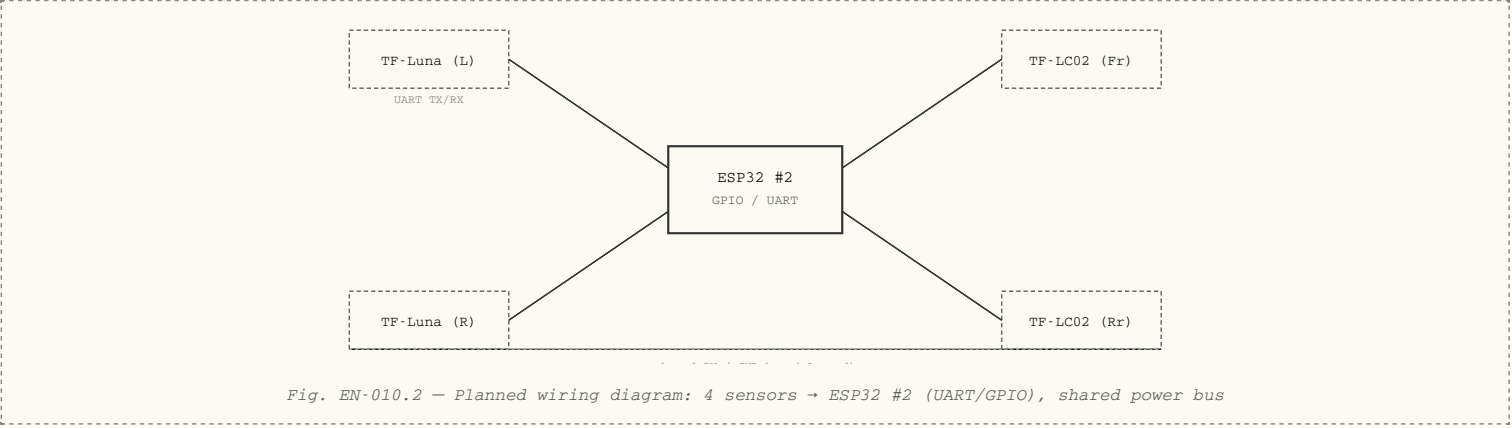
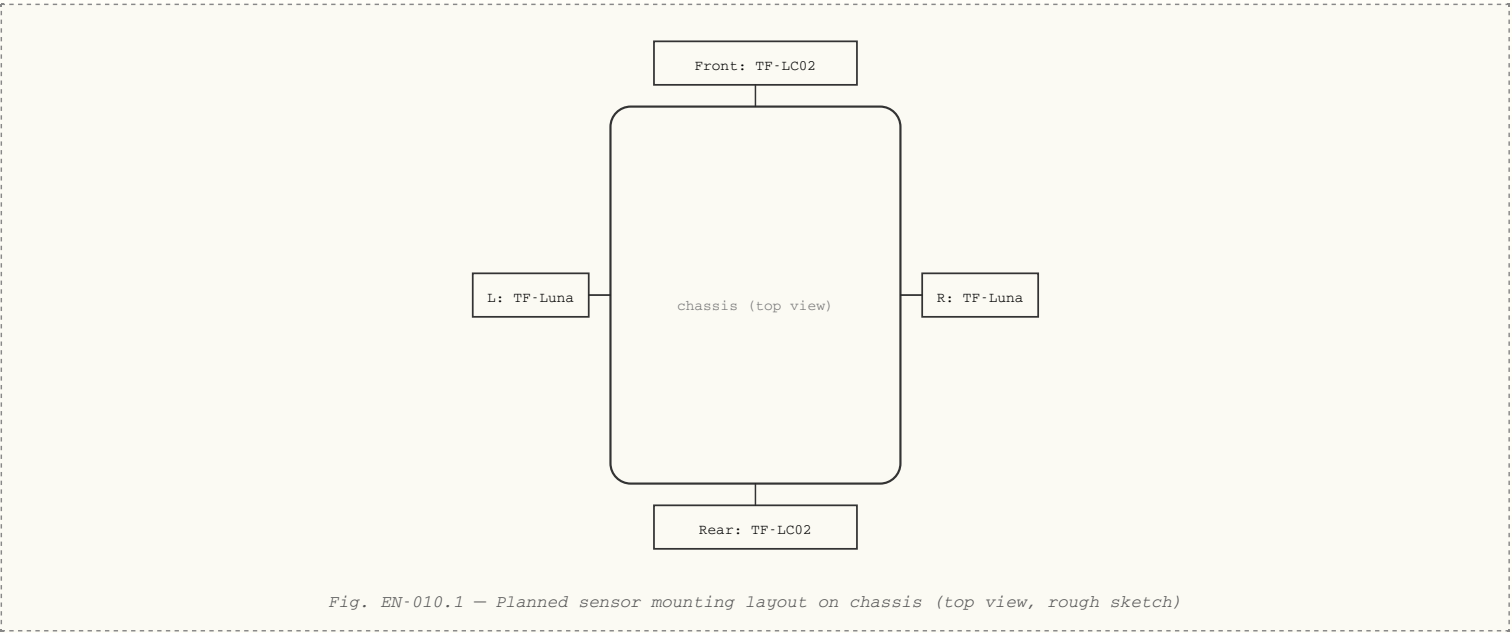
Date: 10 / 07 / 2026

Based on the EN-008 evaluation, the team decided to procure new sensors.

PLANNED CONFIGURATION

Position	Sensor
Left	TF-Luna
Right	TF-Luna
Front	TF-LC02
Rear	TF-LC02

Integration plans, wiring layouts, and communication methods finalized before installation, to reduce implementation time once parts arrived.



EN-011 — Current Development Status

Date: 24 / 07 / 2026

Newly purchased sensors received; integration work about to begin.

CURRENT PRIORITIES

- Mounting new sensors
- Updating firmware
- Integrating sensor data
- Recalibrating wall-following algorithms
- Testing obstacle detection
- Validating parking performance
- Conducting full-track trials

Project is entering its final development phase. Competition date: **26 August 2026**. Remaining work: system integration, testing, optimization, and consistent performance under competition conditions.

Next entry (EN-012) will begin actual TF-Luna / TF-LC02 bench work — individual sensor bring-up before full mounting.

PARTS INVENTORY CHECK

Item	Qty Received	Condition
TF-Luna	2	OK, no visible damage
TF-LC02	2	OK, no visible damage
Second ESP32	1	OK
Jumper wires / connectors	—	Sufficient stock on hand

RISK NOTES

Main risk at this point is time: with the competition on 26 August 2026, there is a limited window to integrate, debug, and re-tune the whole sensing and control pipeline. Plan is to get one sensor talking to ESP32 #2 first, confirm the approach works end-to-end, then repeat for the remaining three rather than wiring all four at once. Old ultrasonic sensors and code path being kept intact and untouched as a fallback until the new sensors are proven reliable on the full track.

DEFINITION OF DONE FOR THIS PHASE

- All four ToF sensors reporting stable readings to ESP32 #2
- Wall-following at least as smooth as the old ultrasonic baseline
- Obstacle detection working at both front and rear
- Parking maneuver attempted successfully at least once end-to-end

OBJECTIVE

Three weeks out from competition, before touching any new hardware, the team paused for a single planning session covering three connected questions: what the robot's real architecture is, where each new sensor should go, and how they'd physically fit — rather than treating these as separate problems solved on separate days.

1. ARCHITECTURE CORRECTION: ESP32 VS. HUB

Early documentation suggested the **ESP32** handled motor control and wall-following directly. On closer inspection this turned out to be wrong: the real navigation brain is the **LEGO SPIKE Hub**, which runs its own PID and steering logic. The ESP32 had only ever been a message-passer between the Raspberry Pi and the Hub.

Correction: SPIKE Hub = navigation brain. ESP32 = bridge only. This single correction reshaped every decision that followed.

2. MATCHING SENSORS TO JOBS

With the architecture clarified, each new sensor was matched to a job based on its actual strengths rather than just its position:

Sensor	Role	Why
TF-Luna (left)	Continuous wall-following	Narrow beam, precise, built for steady side-distance tracking
TF-Luna (right)	Continuous wall-following	Same as left, mirrored
TF-LC02 (front)	Obstacle detection	Close-range precision for spotting obstacles ahead
TF-LC02 (rear)	Parking precision	Close-range precision for nailing the parking maneuver
LEGO Color Sensor	Lap counting	Already reliable reading blue/orange track lines — left untouched

The color sensor was the one part of the original sensing setup that didn't need replacing, so it was left exactly where it was.

3. THE PORT-COUNT PROBLEM

The LEGO Hub simply did not have enough free ports left to take on four new sensors. This turned out to be a bigger decision point than it first looked — since the sensors could not physically connect to the Hub, the wall-following **logic** had to move as well, not just the wiring:

- Wall-following navigation math moved out of the Hub's Pybricks code and into Python on the Raspberry Pi
- The Hub was simplified down to just executing drive/steer commands it receives — no longer doing any sensor reasoning itself
- The ESP32 stayed a simple bridge throughout this change — its role did not need to grow

This built directly on point 1: once it was clear the Hub wasn't the "brain," moving logic off it stopped being a big philosophical change and became just a practical port-count fix. Lesson noted: verify the architecture actually running on the hardware before planning changes on top of it — documentation had drifted from reality.

EN-015 — Getting TF-Luna Talking: UART First, Then I2C

Date: 01 / 08 / 2026

STEP 1: PROVING UART WORKS

Before anything more ambitious, the basic idea needed proving: could the Raspberry Pi actually talk to a TF-Luna at all? The sensor was wired through a TTL-USB adapter for a first bench test.

Result: worked on the first real test. Clean distance readings, changing smoothly as a hand was moved past the sensor.

The test script written for this became the template for all TF-Luna/TF-LC02 reading code that followed.

STEP 2: THE I2C DETECTIVE WORK

UART worked, but used up a full USB port per sensor. To save ports, the next target was getting the TF-Lunas running in I2C mode instead. This hit real friction immediately — the datasheet on hand didn't even show a mode-select pin, and multiple `i2cdetect` scans came back completely empty. Worked through methodically, in order:

1. Confirmed the sensor was actually powered — the small red light inside the lens turned out to be a reliable power-on indicator
2. Confirmed wiring pin-by-pin against the datasheet
3. Confirmed grounds were genuinely shared between the sensor and the Pi

None of that alone fixed it. The breakthrough came from finding a **newer, more detailed datasheet revision** that clearly showed what the older one missed: **pin 5, grounded, switches the sensor into I2C mode**.

Result: one more wire (pin 5 to ground), and the first `10` appeared in the `i2cdetect` scan grid — followed by real, live distance readings.

Lesson noted: when a datasheet doesn't match reality, check for a newer revision before assuming the wiring or the sensor is at fault.

EN-017 — The Address Wall, and the Pivot to a Second ESP32

Date: 02 / 08 / 2026

THE PROBLEM

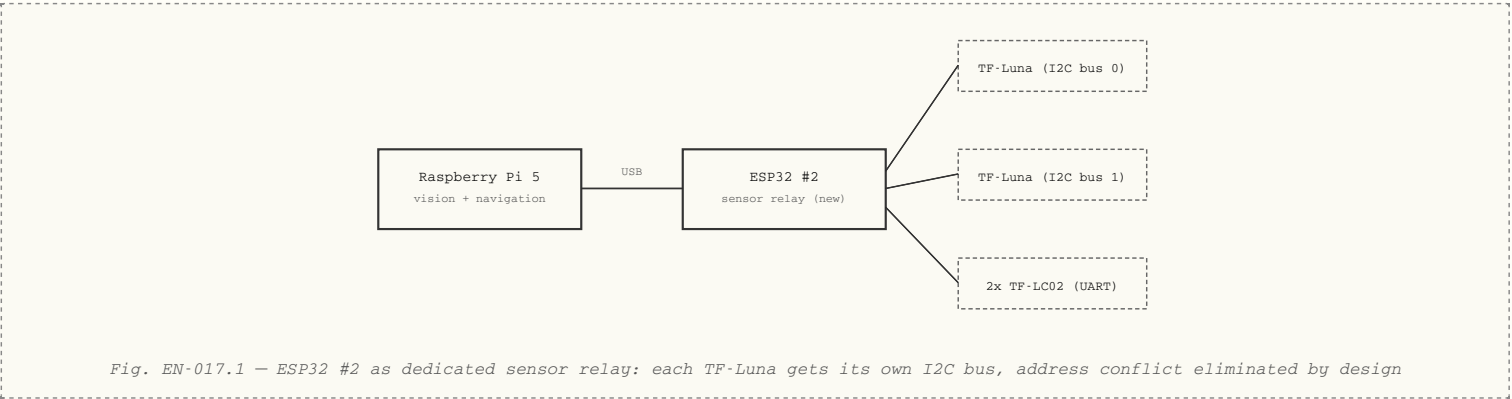
With one TF-Luna working over I2C, the next step was getting a second one onto the same bus — which meant giving it a different address, since both units ship with the same factory default. This is where things got stubborn: the address-change command consistently **reported success but never actually stuck**. The sensor kept answering at its old default address after every attempt, even after retrying with different command ordering and longer waits between steps.

Decision: rather than keep guessing at unfamiliar firmware internals under time pressure, a different path was chosen instead of continuing to fight this one.

THE PIVOT

Instead of continuing to fight the address issue, the decision was made to add a **second ESP32, dedicated purely to sensors**. It reads the second TF-Luna over I2C and both TF-LC02 units over UART, then relays all three readings to the Pi over a single USB serial connection.

This turned out to be a genuinely elegant fix, not just a workaround. Since each TF-Luna now lives alone on its own I2C bus, **neither one ever needs a non-default address again** — the whole address-conflict problem simply disappears rather than getting patched around. As a bonus, this freed up the TTL-USB adapters that had been tied up testing the LC02 units, since both now wire directly into the new ESP32 instead.



EN-019 — Two More Decisions Along the Way

Date: 02 / 08 / 2026

MID-TURN FRONT SENSOR MISREADING

A sharp observation came up during review: a fixed front sensor might misread distances mid-turn, as the robot rotates into a corner and the sensor's beam angle briefly points somewhere misleading.

A servo-mounted sensor was considered but rejected — it would mean more wiring, more mechanical wear, and more code for a problem that didn't need a mechanical solution. Instead, the plan settled on a **software fix**: briefly trusting the front sensor less while the robot is actively turning, rather than reacting to every reading at full weight.

| *Update: this call was later reversed once a servo became available for the sensor cluster — see EN-025.*

FINAL SENSOR PLACEMENT, MAPPED TO THE ROUNDS

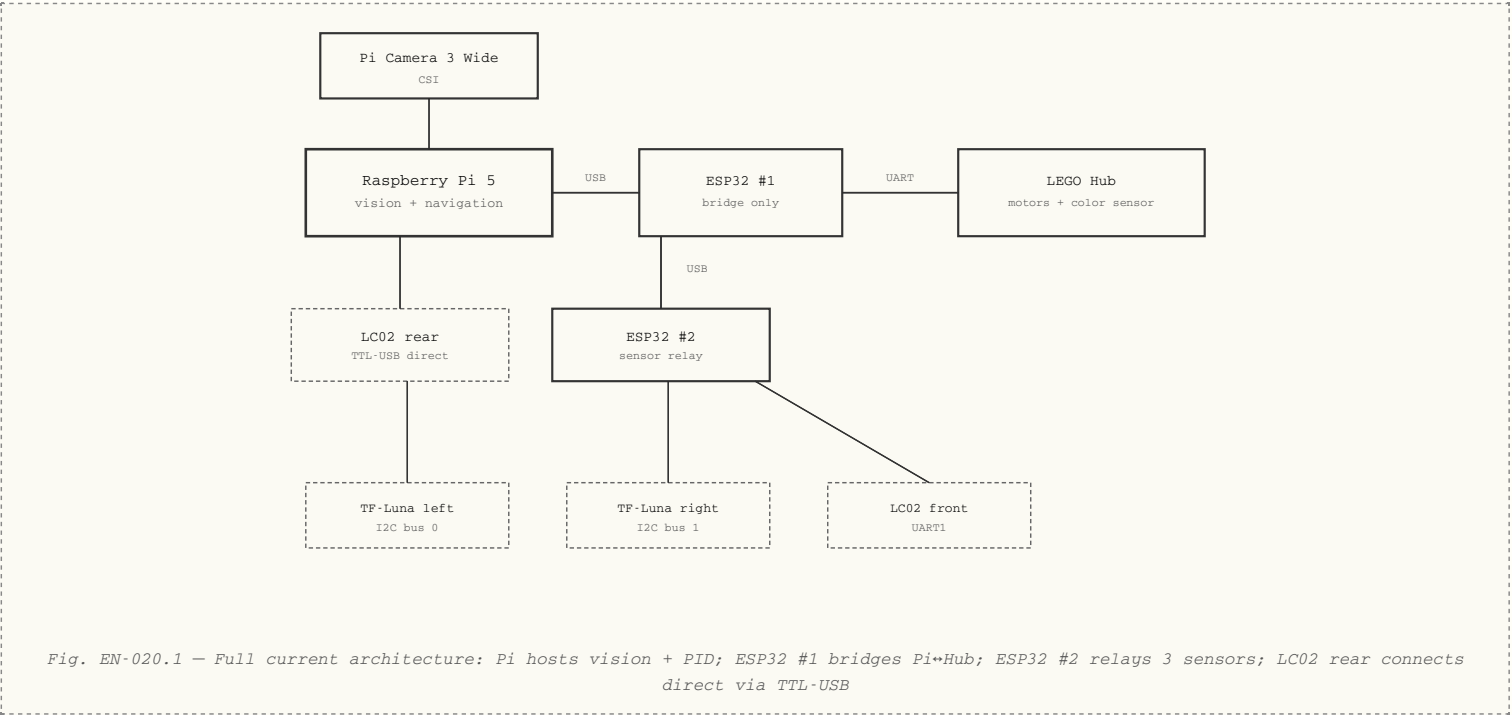
With the sensor lineup settled, final physical placement was mapped directly onto what each competition round actually demands:

- **Side sensors (TF-Luna L/R)** — continuous lap-following, both rounds
- **Front sensor (TF-LC02)** — obstacle detection, primarily round 2
- **Rear sensor (TF-LC02)** — parking precision, specifically for the round 2 parking maneuver

EN-020 — System Architecture, As of Now

Date: 02 / 08 / 2026

With the sensor migration decisions settled (EN-012 through EN-019), this is the complete, current architecture of RT-X7 going into final bench testing and PID tuning.



SUMMARY TABLE

Component	Role	Link
Pi Camera 3 Wide	Vision input	CSI
Raspberry Pi 5	Vision + all navigation/PID logic	—
ESP32 #1	Pure UART bridge, Pi ↔ Hub	USB serial / UART
LEGO SPIKE Hub	Motor execution + color sensor (lap count)	UART
ESP32 #2	Dedicated sensor relay	USB serial to Pi
TF-Luna left	Wall-following (left)	I2C bus 0, via ESP32 #2
TF-Luna right	Wall-following (right)	I2C bus 1, via ESP32 #2
TF-LC02 front	Obstacle detection	UART1, via ESP32 #2
TF-LC02 rear	Parking precision	TTL-USB (FTDI FT232), direct to Pi

Next entries will log actual bench-test results as each sensor is brought online in this final configuration, followed by PID re-tuning on the Pi.

Update: the data-wiring shown here held up, but the physical mounting did not — see EN-025 through EN-027 for the locked, final version.

EN-021 — Idea: Consolidating All 4 Sensors on One ESP32

Date: 03 / 08 / 2026

OBJECTIVE

With ESP32 #2 already relaying three sensors, the question came up: could the same board also take the rear TF-LC02, so every sensor lives on one board instead of splitting one off to a direct USB link?

RESOURCE MATH

Checked against the actual hardware limits of the WROOM-32 38-pin module before committing to anything:

Resource	Total	Used	Free
Hardware UARTs	3	1 (UART0, talks to Pi over USB)	2 — exactly enough for both TF-LC02 units
Hardware I2C buses	2 (I2C0 / I2C1)	0	2 — exactly enough for both TF-Luna units, no address change needed

2 LC02 (UART) + 2 TF-Luna (I2C) = exactly 4 sensors on exactly 4 free hardware channels. Nothing doubled up, nothing emulated in software.

WHAT WOULD CHANGE

- Pi side gets simpler — one line of text from ESP32 #2 instead of parsing a second, independent USB source
- One less USB-adaptor circuit to build, wire, and power

THE HONEST TRADE-OFF

Consolidating everything onto one board means if that ESP32 ever crashes or its USB connection drops mid-run, all four sensors go down at once instead of just three — with the rear sensor on its own line, parking precision would keep working even through a bad moment on ESP32 #2. Noted as a real but small risk, not a blocker.

Decision at this point: lean toward consolidating, pending proof the second I2C bus genuinely works as expected on this exact board. See EN-022 onward.

EN-022 — Verifying the Dual I2C Bus Claim

Date: 03 / 08 / 2026

WHY THIS NEEDED CHECKING

The EN-021 plan depends entirely on the ESP32 having two genuinely independent hardware I2C buses. This got questioned directly — the mentor's understanding was that this board doesn't have a second I2C bus at all. Worth resolving with sources before wiring anything, not just going on memory.

FINDINGS

Checked against Espressif's own datasheet rather than general assumption. Confirmed: the ESP32 has two independent I2C controllers (I2C0 and I2C1), documented as identical in operation and able to run simultaneously. In Arduino code this maps directly — the **Wire** object is the first controller, **Wire1** is the second, and both can run at the same time as fully separate buses.

RECONCILING WITH THE MENTOR'S NOTE

Found the likely source of the mismatch: not every ESP32 variant has two I2C controllers. The original ESP32, ESP32-S, and ESP32-H series have two; the smaller, cheaper **ESP32-C series has only one**. If the mentor's reference point was a C-series board, that advice would have been correct for that chip — just not for the classic WROOM-32 module actually in use here.

Conclusion: solid ground to build on for the WROOM-32. Still to be proven on this specific board, in practice, before fully committing — see EN-023.

Lesson noted: a mentor's caution was correct for hardware they'd worked with before, but didn't transfer directly to this specific module — worth checking which exact chip a piece of advice was based on before applying it.

EN-023 — Proving the Dual I2C Bus, Step by Step

Date: 04 / 08 / 2026

STEP 1: ONE SENSOR FIRST

Before touching the relay firmware or full wiring, the second I2C bus was tested in isolation with a throwaway sketch. One TF-Luna wired to GPIO18 (SDA) / GPIO19 (SCL) — the second bus, **Wire1** — plus 5V/GND and pin 5 to ground as usual. The sketch scans both **Wire** (bus 0, GPIO21/22) and **Wire1** (bus 1, GPIO18/19) every 3 seconds over serial.

Bus	Pins	Result
Wire (bus 0)	GPIO21/22	Nothing found — expected, nothing wired here yet
Wire1 (bus 1)	GPIO18/19	Found device at address 0x10

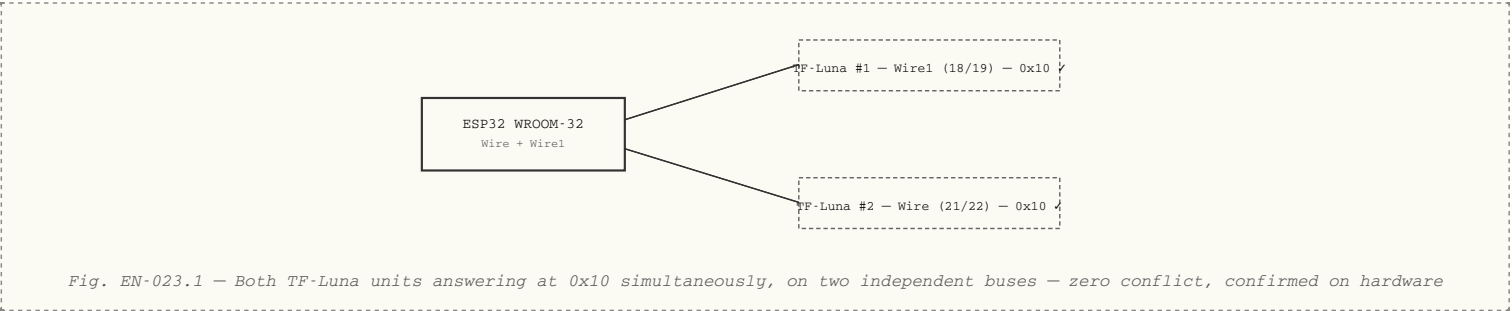
Result: the second I2C bus genuinely works on this board, in practice, not just on paper. Theory confirmed.

STEP 2: BOTH BUSES AT ONCE

Next step defined and carried out: wire the second TF-Luna to the default **Wire** pins and run both sensors at once, to prove there's zero conflict between two identical-address sensors on two separate buses.

TF-Luna #2 Pin	Connects To
Pin 1 (+5V)	5V rail
Pin 2 (SDA)	GPIO21
Pin 3 (SCL)	GPIO22
Pin 4 (GND)	Ground
Pin 5 (mode select)	Ground

Result: both buses reported address **0x10** at the same time, on two different physical sensors, with zero conflict. The dual-bus plan works exactly as designed — no address change ever needed for either TF-Luna.



With this proven, the EN-021 consolidation plan for the two TF-Luna units is locked in: both run on ESP32 #2's two hardware I2C buses, permanently, with no address management required.

EN-025 — Rethinking Sensor Mounting: Servo Cluster, Rear Stays Independent

Date: 04 / 08 / 2026

REVERSING EN-019: A SERVO FOR THE SENSOR CLUSTER

EN-019 rejected a servo-mounted front sensor in favor of a software fix, on the reasoning that a physical solution meant more wiring, wear, and code than the problem justified. With a servo now available and the sensor wiring already being reworked for EN-021-023, the team re-opened that decision.

Left/right TF-Luna and the front TF-LC02 are now mounted together on a single servo-driven cluster that **rotates with the steering**, keeping all three front-facing sensors aligned with the direction of travel through a turn — not just reading less aggressively mid-turn, but pointed the right way to begin with. The servo is driven directly by ESP32 #2, which already owns all three of those sensors.

This supersedes the EN-019 software-only decision. The mid-turn-mistrust logic is no longer the primary fix; the sensor cluster physically tracks the turn instead.

Kept EN-019 in the notebook rather than deleting it — it was the right call with the information and parts on hand at the time. This is a case of the plan evolving as constraints changed, not the earlier reasoning being wrong.

REVISITING EN-021'S FULL-CONSOLIDATION IDEA: REAR TF-LC02 STAYS INDEPENDENT

EN-021 explored putting all four sensors on ESP32 #2. With the servo decision above in place, that plan no longer makes sense for the rear sensor specifically: three sensors now live on a cluster that swings with the steering, close to ESP32 #2, while the rear TF-LC02 sits fixed at the back of the chassis. Routing a wire from a fixed rear point, across the chassis, to a board mounted near a constantly moving part is exactly the kind of wiring strain worth avoiding this close to competition.

Decision: rear TF-LC02 stays on its own direct USB connection to the Raspberry Pi (via FTDI, as originally set up in EN-020) — short, fixed, and untouched by the servo. Only the three servo-mounted sensors consolidate onto ESP32 #2.

Net result: EN-021's "all 4 on one board" idea is **partially adopted** — both TF-Luna and the front TF-LC02 consolidate (3 sensors), rear TF-LC02 stays independent for practical wiring reasons, not reliability ones.

EN-027 — Final Locked Architecture

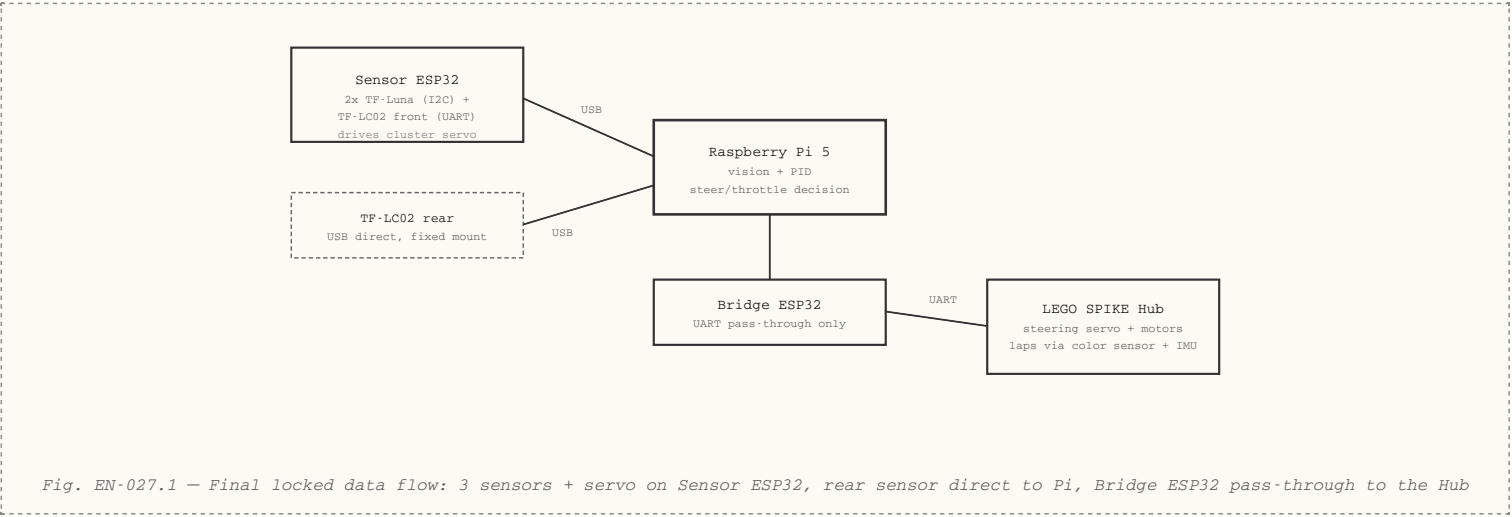
Date: 05 / 08 / 2026

With EN-021 through EN-026 settled, this is the complete, final architecture of RT-X7 going into competition. Superseding EN-020.

COMPONENT LIST

Component	Qty	Notes
LEGO Technic Chassis + SPIKE Prime Hub	1	Execution layer, Ackermann steering, built-in IMU
LEGO Color Sensor	1	Lap counting
Angular servo motor (LEGO-native, steering)	1	Driven by the Hub
Angular servo motor (sensor cluster)	1	Driven by Sensor ESP32; rotates L/R TF-Luna + front TF-LC02 with steering
Raspberry Pi 5	1	Vision + PID logic; reads rear TF-LC02 over USB
Raspberry Pi Camera Module 3 Wide	1	CSI interface
ESP32 Dev Board	2	One Sensor ESP32, one Bridge ESP32
TF-Luna LiDAR	2	I2C, dual independent buses, mounted on sensor-cluster servo
TF-LC02 LiDAR (front)	1	UART to Sensor ESP32, mounted on sensor-cluster servo
TF-LC02 LiDAR (rear)	1	USB direct to Pi, fixed mount
Buck Converter (3518, USB-C QC4.0/PD3.0)	1	Power regulation

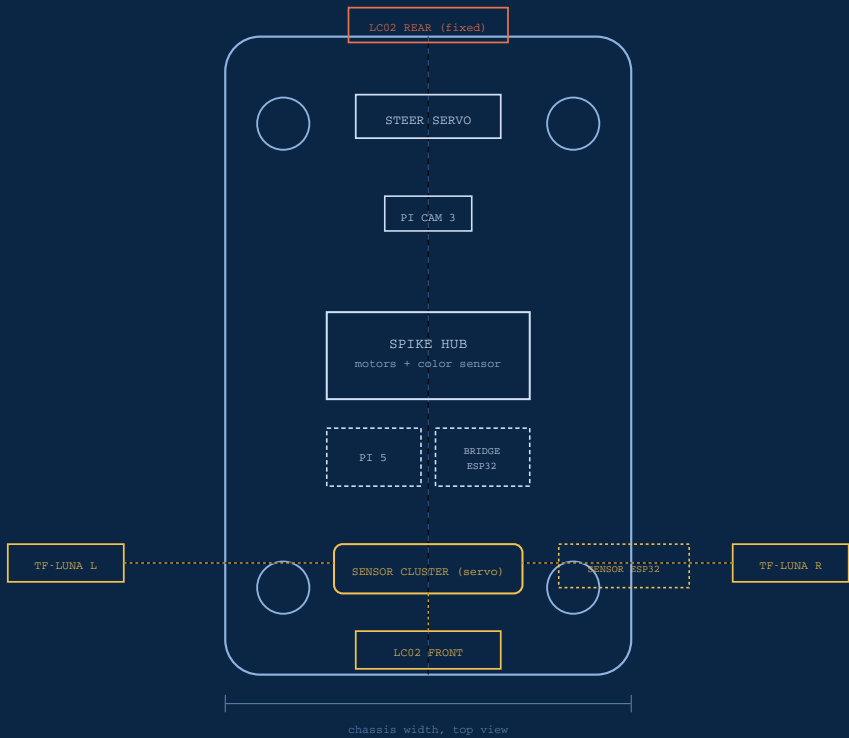
DATA & CONTROL FLOW



Status: LOCKED for competition (26 August 2026). This architecture and component list is the reference version for both the GitHub repository and the printed notebook.

RT-X7 — ROBOT BLUEPRINT

UWR Red Torque • WRO 2026 Future Engineers • Locked Configuration



- Servo-mounted sensor cluster (rotates with steering)
- Fixed rear sensor
- Core compute / drive

Top-down schematic of RT-X7's locked sensor and compute layout. The sensor cluster (yellow) carries both TF-Luna units and the front TF-LC02, rotating with the steering servo so the front-facing sensors stay aligned with the direction of travel through a turn. The rear TF-LC02 (orange) is fixed and wired independently to the Pi for parking precision. The SPIKE Hub remains the sole motor/steering executor; both ESP32 boards are pass-through/relay only — all decision-making lives on the Raspberry Pi.

CHANNEL SUMMARY

Link	Protocol	Endpoint A	Endpoint B
TF-Luna L	I2C (Wire1)	GPIO18/19	Sensor ESP32
TF-Luna R	I2C (Wire)	GPIO21/22	Sensor ESP32
TF-LC02 front	UART	Sensor ESP32	—
TF-LC02 rear	USB (FTDI)	Raspberry Pi 5	— (fixed, independent)
Sensor ESP32 ↔ Pi	USB serial	Sensor ESP32	Raspberry Pi 5
Pi ↔ Bridge ESP32	USB serial	Raspberry Pi 5	Bridge ESP32
Bridge ESP32 ↔ Hub	UART	Bridge ESP32	SPIKE Prime Hub